New York University Tandon School of Engineering
Mechanical and Aerospace Engineering

ROB-GY 6013 Mathematics for Robotics

Problem Set 2: Machine Learning

Due Wednesday, October 21, 2026, before class (5:00 PM)
Submit on Gradescope.

Discussion with other students is allowed. Write your solutions alone.
Use any tool to understand the material, but not to write your solutions.
Solutions typeset in LaTeX or Markdown earn a 10% bonus. Show your reasoning: a bare final answer earns little credit.

Problem 1: Gradients and weighted least squares (20 points)

A linear model predicts a target from a feature vector $x \in \mathbb{R}^2$ by $\hat{y} = x^\mathsf{T} w$. We fit it to three training examples by minimizing the squared error

$$L(w) = \tfrac{1}{2}\|Xw - y\|^2, \qquad X = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad y = \begin{bmatrix} 2 \\ 2 \\ 3 \end{bmatrix},$$

where the rows of X are the three feature vectors. In Problem Set 1 you solved least-squares problems by projection. Here you use calculus instead: the minimum is where the gradient vanishes. Problem 2 uses the same data.

(a) (4 pts) Write $L(w)$ out in terms of w_1 and w_2. Compute the gradient $\nabla L(w)$, and check that it agrees with $\nabla L(w) = X^\mathsf{T}(Xw - y)$.

(b) (5 pts) Find the point w^* where $\nabla L(w^*) = 0$, and compute $L(w^*)$ and the residual $r = y - Xw^*$. Check that r is orthogonal to both columns of X, and explain why $\nabla L(w^*) = 0$ says exactly this.

(c) (6 pts) Suppose the third example comes from a more accurate sensor, so we give its error the weight 4:

$$L_c(w) = \tfrac{1}{2}\Big[(w_1 + w_2 - 2)^2 + (w_1 - 2)^2 + 4\,(w_2 - 3)^2\Big].$$

Compute $\nabla L_c(w)$ and the minimizer w_c^*. Compare the residuals $y - Xw$ at w_c^* with those at w^*, and explain the change.

(d) (5 pts) To keep the weights small, add a penalty: $L_\lambda(w) = L(w) + \frac{\lambda}{2}\|w\|^2$ with $\lambda = 1$. Compute $\nabla L_\lambda(w)$ and the minimizer w_λ^*, and compare w_λ^* with w^*. What happens to w_λ^* as $\lambda \to 0$, and as $\lambda \to \infty$?

Problem 2: Gradient descent, eigenvalues, and step size (20 points)

Solving $\nabla L(w) = 0$ directly means solving a linear system with the matrix $X^\top X$, which is too expensive for large models. Gradient descent instead starts from a guess w_0 and repeatedly steps against the gradient,

$$w_{t+1} = w_t - \eta \nabla L(w_t),$$

with a step size $\eta > 0$. In this problem you run gradient descent on the loss of Problem 1 and find which step sizes make it converge. Let $e_t = w_t - w^*$ be the error, where w^* is the minimizer from Problem 1(b).

(a) (4 pts) Using the gradient from Problem 1(a), write one step as $w_{t+1} = Aw_t + b$. Write out the 2×2 matrix A and the vector b in terms of η, and check that w^* is a fixed point: $Aw^* + b = w^*$.

(b) (5 pts) Start from $w_0 = 0$. Compute w_1 and w_2 for $\eta = \frac{1}{2}$ and for $\eta = 1$, together with the errors e_0, e_1 and e_2. What do you observe?

(c) (6 pts) Because w^* is a fixed point, the error obeys $e_{t+1} = Ae_t$. Find the eigenvalues and eigenvectors q_1, q_2 of $X^\top X$. Compute Aq_1 and Aq_2, and give the eigenvalues of A in terms of η. Write e_t as a combination of q_1 and q_2, as in Problem 2 of Problem Set 1, and find all step sizes η for which $e_t \to 0$ from every starting point. Which eigenvalue of $X^\top X$ limits the step size?

(d) (5 pts) Use (c) to explain what you observed in (b). Why does $\eta = 1$ diverge, and why does its error point along a single direction after one step? Which step size makes the error shrink fastest, and by what factor per step?

Problem 3: Backpropagation through a two-layer network (20 points)

A two-layer network with a ReLU activation maps an input x to a prediction

$$z = W_1 x + b_1, \qquad a = \mathrm{ReLU}(z), \qquad \hat{y} = W_2 a + b_2,$$

where $\mathrm{ReLU}(t) = \max(0, t)$ acts on each entry, and x, z and a are column vectors (as are their gradients). The network is trained on one example (x, y) with the loss $L = \frac{1}{2}(\hat{y} - y)^2$. Gradient descent needs the derivative of L with respect to every parameter, and backpropagation computes all of them in one backward pass using two rules:

- For a single-example linear layer $z = Wx + b$ with upstream gradient $g_z = \nabla_z L$: $\nabla_W L = g_z x^\top$, $\nabla_x L = W^\top g_z$, and $\nabla_b L = g_z$. Matrix gradients have the same shape as their parameters.
- For $a = \mathrm{ReLU}(z)$: $\nabla_z L = (\nabla_a L) \odot \mathbb{1}[z > 0]$, where $\odot$ multiplies entrywise and $\mathbb{1}[z > 0]$ is 1 where $z_j > 0$ and 0 elsewhere.

Use the parameters and training example

$$x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad y = 1, \quad W_1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 2 & 1 \end{bmatrix}, \quad b_2 = -1.$$

(a) (4 pts) Compute the forward pass: z, a, $\hat{y}$ and L.

(b) (7 pts) Compute the backward pass, in this order: $\partial L/\partial \hat{y}$, $\nabla_{W_2} L$, $\partial L/\partial b_2$, $\nabla_a L$, $\nabla_z L$, $\nabla_{W_1} L$ and $\nabla_{b_1} L$. State which rule you use at each step.

(c) (5 pts) Which entries of the gradients in (b) are zero, and why? There are two different reasons. Does $\nabla_{W_1} L$ depend on W_2? On y? Answer from the backward pass.

(d) (4 pts) Suppose the ReLU is removed, so that $a = z$. Multiply out the two layers to write $\hat{y} = Wx + c$ with a single matrix W and a number c, and compute W and c for the parameters above. What does this say about stacking linear layers without an activation?

Problem 4: Computation graphs and automatic differentiation (20 points)

In Problem 3 you applied the chain rule by hand, one layer at a time. Libraries such as PyTorch do this automatically. During the forward pass they record every elementary operation, such as an addition or a multiplication, in a computation graph. During the backward pass they visit the graph in reverse order and apply the chain rule one operation at a time. In this problem you complete a small engine of this kind, and use it to check Problem 3 and to train a network. The engine follows Andrej Karpathy's micrograd. The first video of his series *Neural Networks: Zero to Hero*, at `https://karpathy.ai/zero-to-hero.html`, builds it step by step and is optional background.

The starter notebook `ps2-autograd.ipynb` defines a class `Value`. A `Value` stores a number `data` and a number `grad`, and it remembers the `Value`s it was computed from. After `L.backward()`, every `Value` `v` that `L` depends on has `v.grad` equal to $\partial L/\partial v$. For example,

```
a = Value(2.0)
b = Value(-3.0)
L = a * b + 1       # L.data is -5.0
L.backward()        # now a.grad is -3.0 and b.grad is 2.0
```

since $\partial L/\partial a = b$ and $\partial L/\partial b = a$. To make this work, every operation stores a small function `_backward`. It adds the local derivatives of the operation, multiplied by the gradient `out.grad` of its output, to the `grad` of each input. Addition and `backward()` are already complete. The notebook also provides helper functions that build networks out of `Value`s and compute the loss $L = \frac{1}{2n}\sum_{k=1}^{n}(\hat{y}_k - y_k)^2$ over n examples. For one example this is the loss of Problem 3.

Write code only in the notebook cells marked `TODO`, and answer the questions of (a) and (d) in the text cell at the end of the notebook. Submit the notebook as a PDF with all code and outputs visible. Do not use PyTorch or any other automatic-differentiation library.

(a) (3 pts) Complete `_backward` for multiplication and for ReLU, following the example of addition. The test cell should then print `-3.0 2.0` and `0.0 1.0`. Why does `_backward` use `+=` rather than `=`?

(b) (5 pts) Using `Value`s, write out the forward pass of the network of Problem 3 on its training example, computing $z_1, z_2, a_1, a_2, \hat{y}$ and L one line at a time. Call `backward()` once, and check that the printed gradients of all nine parameters agree with your answer to Problem 3(b).

(c) (6 pts) Check the engine with centred finite differences. For the random network and data in the notebook, first fill in every `grad` with one call to `backward()`. Then, for each parameter θ, change only θ to compute

$$\hat{g} = \frac{L(\theta + h) - L(\theta - h)}{2h}, \qquad h = 10^{-6},$$

and restore θ afterwards. Report the largest relative error

$$\frac{|g - \hat{g}|}{\max\big(|g| + |\hat{g}|,\ 10^{-12}\big)}$$

over all parameters, where g is the gradient from `backward()`. It should be below 10^{-8}.

(d) (6 pts) Train the network with layer sizes 1, 16, 1 in the notebook on the 20 given samples of $\sin x$, $x \in [-\pi, \pi]$, by gradient descent with step size 0.1 for 1000 steps. In every step, reset all gradients to zero, compute the loss, call `backward()`, and update every parameter by $\theta \leftarrow \theta - 0.1\,\partial L/\partial\theta$. Plot the loss and the fitted curve with the functions provided. The final loss should be below 0.01. Then train a fresh copy of the same network with step size 0.5 for 5 steps, and print the loss at every step. What happens? Explain it using Problem 2.